APOSTLE PAUL MEMORIAL PARK · ARINGAY, LA UNION

Mobile companion app

The specification and the work breakdown for the customer-facing mobile app — online reservation, online booking, and the 3D park views — planned as an eight-week build on top of the existing Convex backend.

Planning artifact · not yet built

_bmad-output/planning-artifacts/

PART ONE

Mobile App Specification — Apostle Paul Memorial Park

What the app is, how it fits the backend that already exists, and what has to be built to support it.

Product: *Apostle Paul Memorial Park* companion app (iOS + Android) **Client:** Cases Land Inc. · Zone 1, San Eugenio, Aringay, La Union 2503 **Backend:** existing Convex deployment `beaming-boar-935` (no new backend service) **Timeline:** 8 weeks (target, not a hard deadline — overflow is flagged in §9) **Status:** Draft for review · authored 2026-08-20 **Relationship to existing docs:** extends prd.md and architecture.md; supersedes nothing.

---

0. How to read this document

This is a **build spec**, not a pitch. It assumes the reader has the repo checked out.

Every claim about what already exists is a citation into the codebase — where you see a file path, the thing is real and working today. Where you see **NEW**, it does not exist and someone has to write it.

The three features you asked for are specified in §4 (reservation), §5 (booking), and §6 (3D). The additions I recommend are in §7, ranked and scoped honestly against the 8 weeks. §9 is the week-by-week plan. §10 is what will hurt.

Decisions locked before drafting:

DECISION	CHOICE	CONSEQUENCE
Platform	Expo / React Native, shipped to both app stores	Real push, camera, GPS, offline. Costs ~2 weeks of shell + store setup.
Primary user	Customers and families	Staff keep the existing web app. No role-switching tab bar in v1.
Timeline posture	8 weeks is a **target**	Full feature set planned honestly; what does not fit is marked *Overflow*, not silently cut.

1. Product summary

A family who buys a lot at Apostle Paul Memorial Park today interacts with the park through the office: they visit, they talk to a consultant, they pay at a counter, and they phone ahead to arrange an interment. The web portal (`src/app/(customer)/portal/`) already lets them *look* at what they own — contracts, payments, receipts — and pay an installment online. It does not let them *do* anything new.

The mobile app closes that gap. It is the first surface where a family can:

1. **See the park in three dimensions** and understand where a lot actually sits, before ever driving to Aringay.
2. **Hold a lot** — a real, time-boxed, deposit-backed reservation that takes the lot off the market while the family decides.
3. **Request an interment or memorial service** against the park's real calendar, with the same double-booking guard the office staff rely on.
4. **Find a grave and walk to it**, which matters most on the two days of the year when the park is full of people who have never been there before.

Voice, throughout: **Reverent · Compassionate · Permanent · Restrained.** No exclamation marks, no urgency language, no "limited slots", no countdown timers styled as pressure. A memorial park app that reads like a hotel booking funnel is a failure regardless of how well it works.

---

2. What already exists (and what does not)

This is the most important section for estimating. The backend is far more complete than a greenfield mobile project would assume — and it has one large, specific hole.

2.1 Ready to consume as-is

CAPABILITY	WHERE IT LIVES	MOBILE USE
Customer identity + auth	`convex/auth.ts` (Convex Auth, Password provider); role `"customer"` in `convex/lib/auth.ts:53`	App login
Portal invite redemption	`convex/portalInvites.ts` — `createPortalInvite`, `acceptPortalInvite`	First-run onboarding via emailed link
Own contracts / payments / receipts	`convex/portal.ts` — `listCustomerContracts`, `getCustomerContractDetail`, `listCustomerPayments`, `listCustomerReceipts`	"My estate" tab, zero new backend
Receipt PDF access	`convex/portal.ts` — `requestCustomerReceiptPdf`, `getCustomerReceiptPdfUrl`	Download / share a BIR receipt
Online payment	`convex/portal.ts:1792` `createGatewayPaymentIntent` + `paymentIntents` table (gcash / maya / card) + HMAC webhooks at `https://beaming-boar-935.convex.site/api/{gcash,maya,card}-webhook`	Deposit capture **and** installment payment
Lot geometry, from day one	`lots.geometry` — centroid, polygon vertices, indexed bbox (`convex/schema.ts:208`, ADR-0008)	3D scene, map, wayfinding — no data migration
Viewport lot query	`convex/lots.ts:782` `listInBbox`	Map panning without loading 2,000 lots
Grave lookup	`convex/search.ts:172` `findGrave`	Find-a-Grave
Double-booking guard	`convex/lib/scheduling.ts:109` `assertNoBookingConflict` — throws `SCHEDULING_CONFLICT` on lot / chapel / pathway overlap	Reused verbatim by online booking
Ceremony model	`ceremonies` table — kinds `consecration`, `interment`, `memorial_anniversary`; `chapelReserved` / `pathwayReserved` flags	The thing being booked
Working 3D scene	`src/components/Phase3DMap/Phase3DMap.tsx` — three.js `0.160`, OrbitControls, live lots from `lots:listLots`, per-status colouring, selection rail	**Port target**, not a rewrite

CAPABILITY	WHERE IT LIVES	MOBILE USE
Reminder plumbing	`reminderConfig`, `reminderDeliveries`, `emailReminderLog`, `smsReminderLog`, `convex/crons.ts`	Push becomes a fourth delivery channel
Family estates	`familyEstates` table — 2–12 lots, primary + secondary owners	Multi-heir access in the app
Sections registry	`sections` table — `displayName`, `kind`, `descriptionMarkdown`	Wayfinding labels and section copy
Brand tokens	`tailwind.config.ts`, `apostle-paul-brand-guidelines.html`	Ported to an RN theme module

2.2 The hole

There is no customer-initiated write path for reservation or booking. Specifically:

- `convex/lots.ts:606` `setLotStatusReserved` is gated `requireRole(["admin","office_staff"])`, and its own docstring calls it an *"AC5 smoke-test mutation"* — "the real reservation flow (with deposit capture, contract creation, etc.) lives in Story 3.x". That story was never built for customers.
- There is **no reservation / hold table**. `lots.status` carries a `"reserved"` literal, but nothing records *who* holds it, *until when*, or *against what deposit*. A hold that cannot expire is a lot permanently lost from sellable inventory.
- `convex/ceremonies.ts:173` `scheduleCeremony` is `requireRole(["admin","office_staff"])`. A customer cannot call it — and should not. Customers *request*; staff *confirm*.
- `convex/portal.ts` exposes exactly two customer mutations that change anything: `updateCustomerContact` and `createGatewayPaymentIntent`. Everything else is read-only.

Consequence for the plan: roughly 35% of the 8 weeks is Convex backend work, not React Native work. Any estimate that treats this as "wrap the portal in an app" is wrong by about three weeks. The new backend surface is specified in §8.

2.3 Conventions the mobile client must honour

Non-negotiable; enforced by lint and review.

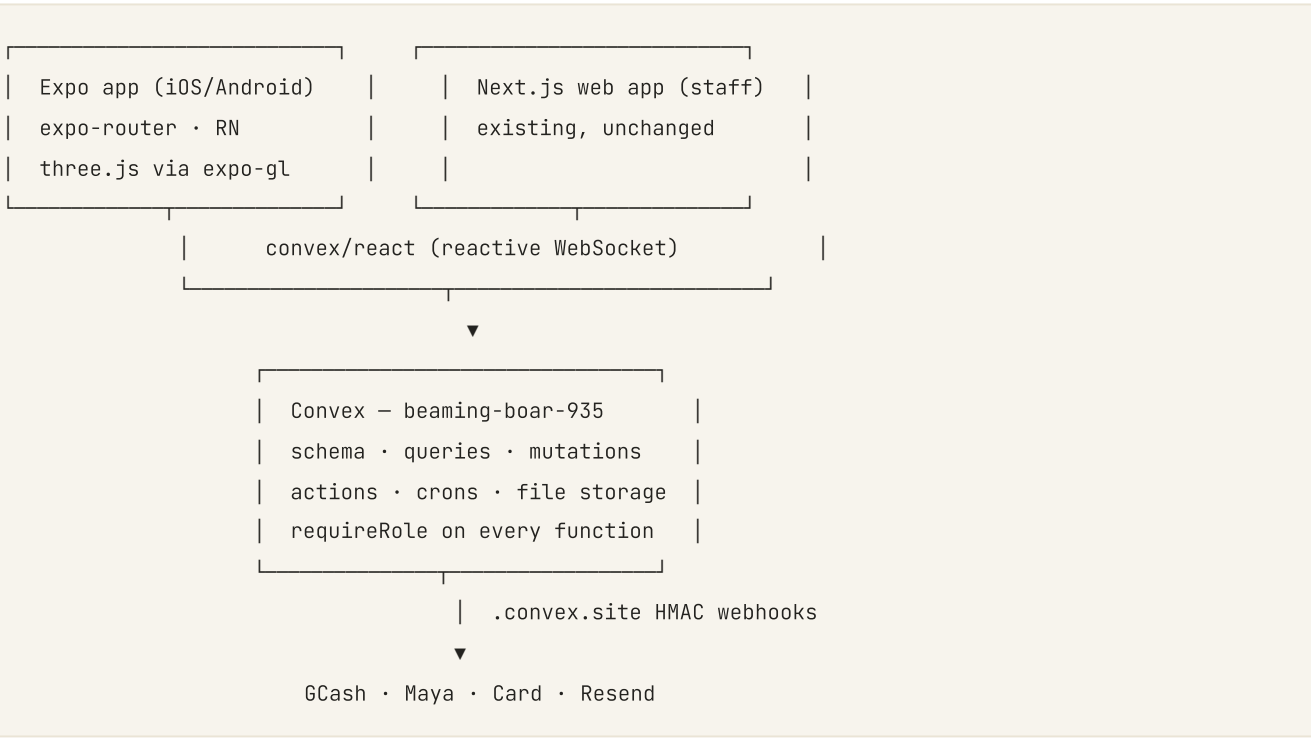
- **`convex/_generated` is not committed.** The web client calls Convex through `useQuery(makeFunctionReference<"query">("module:fn"))`. The mobile client does the same. Do not introduce a codegen step for the app alone.
- **Every public Convex function's first awaited line is `await requireRole(ctx, [...])`** — enforced by `local-rules/require-role-first-line` (`eslint-local-rules.js`). New mobile-facing functions are not exempt.
- **Money is integer centavos** (ADR-0007), formatted client-side. Port `formatPeso` from `src/lib/money`; do not reimplement rounding.
- **Ownership scoping is server-side and derived.** Handlers never accept a `customerId` from the client — it is resolved from the authenticated email (`convex/portal.ts` → `resolveCurrentCustomer`). Non-owned reads return `null`, not `FORBIDDEN`, so existence does not leak. The app inherits this exactly.

- **Every mutation emits an audit row** via `emitAudit` (ADR-0004). Customer-initiated mutations are the *most* likely to be disputed later, so they are the least exempt.
- **Status changes go through the state machine** (ADR-0006), never a raw `db.patch`.
- **`noUncheckedIndexedAccess` is on.** Array index access is `T | undefined`. Carry the same `tsconfig` strictness into the app.

---

3. Architecture

3.1 Shape



One backend, two clients. No BFF, no REST layer, no GraphQL. The app is a second consumer of the same Convex functions, which is the entire reason the stack was chosen (architecture.md §7). When a staff member confirms a booking on the web app, the family's phone updates over the same WebSocket — no polling, no cache-invalidation code.

3.2 Stack

CONCERN	CHOICE	NOTE
Runtime	Expo SDK (current stable) + React Native	Managed workflow; EAS Build for binaries
Routing	`expo-router`	File-based; mirrors the App Router model the team knows
Data	`convex/react` + `ConvexProvider`	Same `useQuery` / `useMutation` hooks as web
Auth	`@convex-dev/auth/react` with `expo-secure-store` token storage	**Spike in week 1** — see §10 R1
3D	`expo-gl` + `expo-three` + `three@0.160` (version-matched to web)	Same version as `package.json` keeps scene code portable
Gateway checkout	`expo-web-browser` `openAuthSessionAsync`	Hosted GCash / Maya checkout, returns via deep link
Push	`expo-notifications` + Expo Push Service	New `pushTokens` table + a fourth delivery channel

CONCERN	CHOICE	NOTE
2D map	`react-native-maps`	Satellite tiles + lot polygons from `lots.geometry`
Offline	Convex client cache + `expo-sqlite` snapshot of owned lots / receipts / map tiles	Poor signal at the park is the norm, not the edge case
Styling	RN `StyleSheet` + ported token module	No NativeWind — one token source (`tailwind.config.ts`) transpiled to a TS object
Distribution	EAS Build → TestFlight + Play Internal Testing → production	

3.3 What we deliberately do not do

- **No WebView wrapper.** The 3D display and the offline map are the two features that justify a native app; a WebView delivers neither well, and Apple reviews thin wrappers harshly (Guideline 4.2).
- **No separate mobile API.** Every temptation to "just add an endpoint for the app" becomes instead a Convex function both clients can use.
- **No duplicated business logic.** Booking-conflict rules, money math, and state transitions stay in `convex/lib/`. The app renders; it does not decide.
- **No staff features in v1.** A field app for groundskeepers is a real and valuable product — and a separate spec.

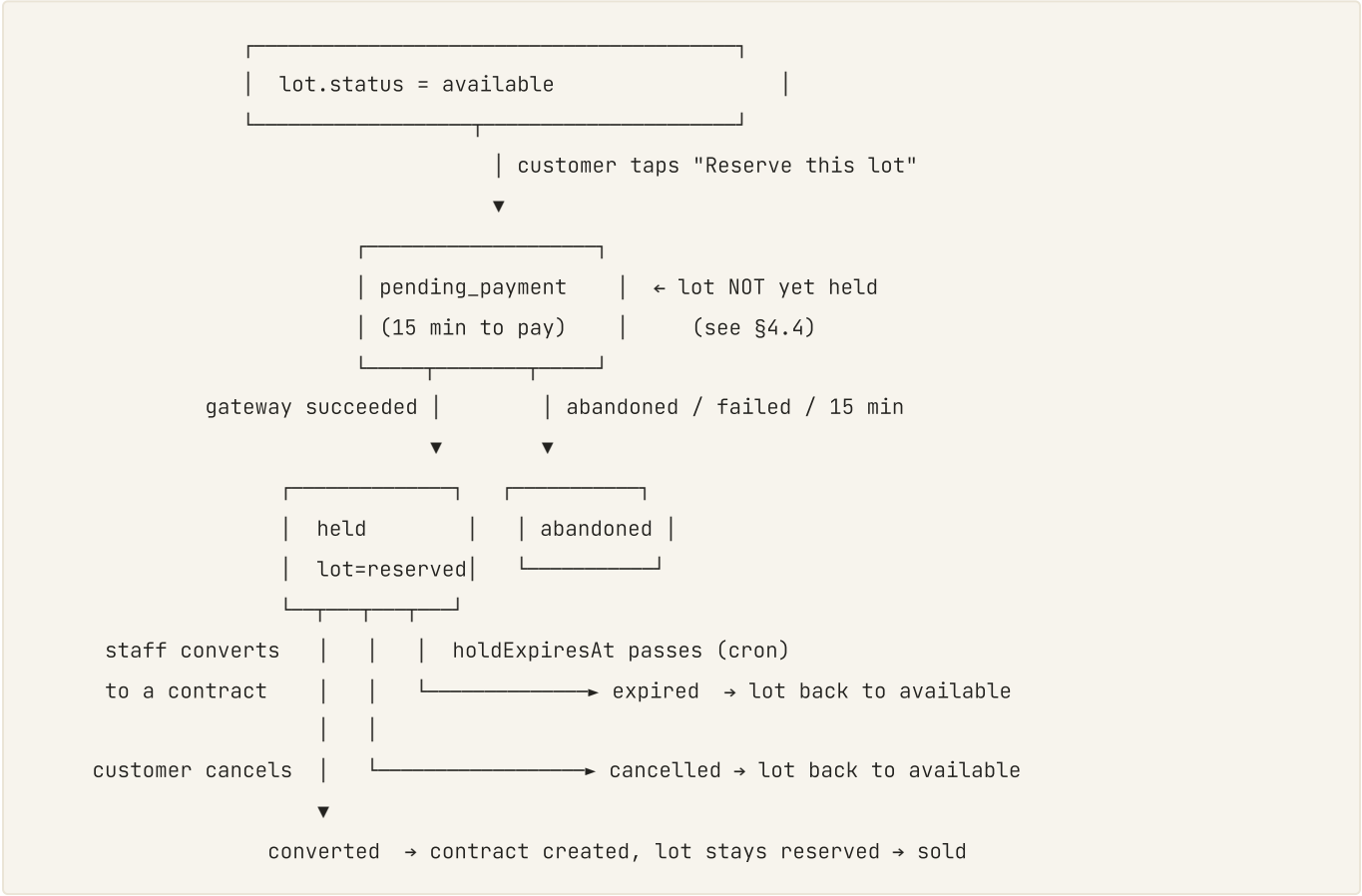
4. Feature: Online Reservation

4.1 What it is

A family browsing the park can place a **time-boxed hold** on an available lot, backed by a reservation fee paid through GCash or Maya. The hold takes the lot out of inventory, gives the family a defined window to complete the purchase with a consultant, and expires automatically if they do not.

This is the feature with the most new backend and the most ways to go wrong. A hold that never expires silently destroys inventory. A hold that expires while the family is mid-payment destroys trust. Both failure modes are addressed below.

4.2 Reservation lifecycle



Five terminal-or-active states: `pending_payment`, `held`, `converted`, `expired`, `cancelled`, plus `abandoned` for intents that never paid. Every transition writes an audit row.

4.3 Rules

RULE	VALUE	RATIONALE
Hold duration	**14 days** from successful payment	Long enough for a family to consult relatives; short enough that inventory turns. *Confirm with the client — see §12 Q1.*

RULE	VALUE	RATIONALE
Reservation fee	Fixed peso amount, configurable in `appSettings`	Not a percentage — families should see a round, predictable number
Fee on conversion	Credited in full against the contract's first payment	Never a "booking fee" the family loses by buying
Fee on expiry / cancellation	**Policy decision required** — see §12 Q1	Refundable, partially refundable, and forfeit are all defensible; the client must choose before build
Concurrent holds per customer	Max 3 active	Prevents inventory hoarding without insulting a family buying an estate
Eligible lots	`status === "available"` and `isRetired === false` only	
Extension	Staff-only, once, +7 days, reason required	Compassionate exception path without a customer-facing loophole
Family estates	v1 reserves **single lots only**	Multi-lot estate holds are Overflow — `familyEstates` requires 2–12 lots atomically held, which is a materially harder mutation

4.4 The race condition, and how it is handled

Two families tapping "Reserve" on lot A-12-3 within the same second is the defining correctness problem of this feature.

The approach: the lot is not held at intent-creation time. It is held at webhook-confirmation time, inside a single Convex mutation that re-reads the lot and re-checks its status before transitioning. Convex mutations are serializable transactions — the second family's mutation observes `status === "reserved"` and fails cleanly.

Because of that ordering there is a real window where two families can both be paying for the same lot. That window is handled, not wished away:

1. The app shows the lot's live status reactively while the checkout sheet is open — if someone else completes first, the family sees it change before they pay.
2. If a payment nonetheless lands on an already-held lot, the mutation marks the reservation `abandoned` with `failureReason: "lot_taken"` and **immediately schedules a refund action**, plus an in-app notice written in the park's voice ("The lot was taken moments before your payment completed. Your reservation fee is being returned in full.").
3. The 15-minute `pending_payment` ceiling keeps the window small.

Alternative considered and rejected: holding the lot optimistically at intent creation. It eliminates the double-payment window but lets any user with a script take the entire park out of inventory for 15 minutes at a time, with no payment required.

4.5 Screens

SCREEN	CONTENT
Browse	Filter by section, lot type (`single` / `family` / `mausoleum` / `niche`), price band, availability. Reads `lots:listLots`.

SCREEN	CONTENT
Lot detail	3D view of the lot in situ (§6), section description from `sections.descriptionMarkdown`, dimensions, price, status pill.
Reserve — review	Lot summary, fee, hold duration, the refund policy in plain language, an explicit consent checkbox. No dark patterns, no scarcity copy.
Reserve — pay	GCash / Maya / card selection → `expo-web-browser` hosted checkout → deep-link return.
Reserve — result	Success: hold confirmed, expiry date, what happens next, consultant contact. Failure: plain explanation and a retry.
My reservations	Active holds with days remaining, history of past holds.

4.6 Acceptance criteria

- **AC1** A customer can reserve an `available` lot end-to-end and the lot's status becomes `reserved` only after gateway confirmation.
- **AC2** Two concurrent reservations on one lot result in exactly one `held` reservation; the loser is `abandoned` with a refund scheduled and a plain-language notice.
- **AC3** A hold past `holdExpiresAt` is released by cron within 15 minutes, and the lot returns to `available`.
- **AC4** A customer can cancel their own hold; the lot returns to `available` immediately.
- **AC5** A customer cannot reserve a lot that is `reserved`, `sold`, `occupied`, or retired — enforced server-side, not merely hidden in the UI.
- **AC6** Every state transition emits an audit row naming the actor.
- **AC7** Staff see all active holds in the web app and can convert one to a contract.
- **AC8** A customer never sees another customer's reservation, including via a crafted id.

5. Feature: Online Booking

5.1 What it is

A family can **request** an interment, a consecration, or a memorial anniversary service against the park's real availability. Staff confirm it. On confirmation the existing `ceremonies` row is created through the existing guarded path, and the family is notified.

Request, not self-serve. An interment involves grave preparation, a tent, a consultant, sometimes a priest, and a family in grief. Letting an app write directly into the ceremony calendar with no human in the loop is the wrong design for this business — and it would bypass `scheduleCeremony`'s deliberate `admin` / `office_staff` gate rather than respect it.

5.2 Flow

```
Customer                        System                          Staff
────────                        ──────                          ─────
picks kind + lot          →   validates ownership
picks a date              →   getAvailableSlots(date)
                              ├─ existing ceremonies that day
                              ├─ chapel / pathway occupancy
                              └─ park operating hours + blackouts
sees open slots           ←   slot list with capacity flags
picks slot, adds notes    →   requestCeremony()
                              ├─ soft conflict pre-check
                              ├─ status: "pending_review"
                              └─ notify office              →   request appears in queue
                                                                staff reviews, may adjust
                          ←   push: "confirmed" or "adjusted"  ←  confirmCeremonyRequest()
                              └─ calls scheduleCeremony()
                                 └─ assertNoBookingConflict()  ←  the load-bearing check
```

The authoritative conflict check runs **at confirmation**, inside the existing `scheduleCeremony` path (`convex/ceremonies.ts:266` → `assertNoBookingConflict`). The check at request time is advisory only — it stops obvious clashes early without becoming a second source of truth that can drift from the first.

5.3 Rules

RULE	VALUE
Who may request	Customer with an active contract on the lot, or a secondary owner on the lot's `familyEstate`
Ceremony kinds	`interment`, `consecration`, `memorial_anniversary` — the existing schema union, unchanged
Default durations	Reuse `CEREMONY_DEFAULT_DURATION_MINUTES` (`convex/ceremonies.ts:77`); bounds 30–240 min (`convex/lib/scheduling.ts:62`)
Lead time	Interment: minimum 24 h. Others: minimum 72 h. *Confirm — §12 Q2*

RULE	VALUE
Chapel / pathway	Requestable as add-ons; capacity resolved by `assertNoBookingConflict`
Staff response target	Surfaced in-app as "The Estate Office will confirm within one business day"
Cancellation	Customer may cancel a `pending_review` request freely; a confirmed ceremony requires contacting the office
Blackout dates	**NEW** `appSettings` entry — Nov 1–2 (Undas) and park-defined closures

5.4 Screens

Request kind → lot picker (only lots the family holds) → calendar month view with availability density → slot picker → add-ons and notes → review → submitted. Then a request-status screen, and a ceremony detail screen once confirmed with date, time, section wayfinding, and an add-to-device-calendar action.

5.5 Acceptance criteria

- **AC1** A customer can only request a ceremony on a lot they own or co-own; server-enforced.
- **AC2** The slot list never offers a window that `assertNoBookingConflict` would reject at the same instant.
- **AC3** A request creates a `pending_review` row and notifies staff; it does **not** create a `ceremonies` row.
- **AC4** Staff confirmation routes through the existing `scheduleCeremony`; on `SCHEDULING_CONFLICT` the request is returned to the family with a plain-language explanation and alternative slots.
- **AC5** Confirmation, adjustment, and decline each push-notify the family.
- **AC6** A request inside the lead-time floor or on a blackout date is refused server-side.
- **AC7** Requests and confirmations both emit audit rows.

6. Feature: 3D displays

6.1 What it is

The park, rendered as an explorable three-dimensional scene, so a family can understand where a lot sits — its section, its neighbours, its distance from the chapel and the gate — without standing in it.

This is the feature that most justifies a native app, and the one with the largest gap between "a demo that impresses" and "a thing that runs at 60fps on a mid-range Android phone in a province with 4G".

6.2 Starting point

`src/components/Phase3DMap/Phase3DMap.tsx` already does this on the web: a three.js scene with OrbitControls, lots extruded from live inventory via `lots:listLots`, coloured by status, with selection, filter chips, a per-section roll-up, and DOM section labels. It was written imperatively inside a single mount effect specifically because WebGL has no React reconciler — which is exactly the structure that ports cleanly to `expo-gl`.

Port, do not rewrite. The plan is to extract the scene-construction logic into a platform-neutral module and keep two thin shells:

```
src/lib/scene3d/            ← NEW shared module, platform-neutral three.js
  buildParcelScene.ts         geometry from lots[], materials, camera rig
  statusMaterials.ts          brand palette → three.js materials
  pickLotAtPointer.ts         raycasting
        |
        ├→ web:    Phase3DMap.tsx           (canvas + DOM labels)
        └→ mobile: ParcelScene.native.tsx   (expo-gl GLView + RN overlay)
```

Two differences that are not cosmetic:

1. **Labels.** The web version styles DOM nodes via `.phase3d-seclabel`. React Native has no DOM — section labels become either `THREE.Sprite` textures inside the scene or absolutely-positioned RN `<Text>` driven by projected coordinates. Sprites are recommended: they occlude correctly and cost one draw call each.
2. **Controls.** OrbitControls binds DOM mouse and touch events. Mobile needs a gesture handler (`react-native-gesture-handler`) driving the same camera rig — one-finger orbit, two-finger pan and pinch-zoom, double-tap to frame a lot.

6.3 Three views, one scene module

VIEW	PURPOSE	CAMERA
Park overview	Whole park, sections colour-coded, phase boundaries	High orbit, auto-rotate at rest
Section view	One section, individual lots, status colours, tap to select	Mid orbit, bounded to the section
Lot in situ	The single lot highlighted among its neighbours, with the marker or monument massed in	Low orbit, framed on the lot

The lot-in-situ view is the one that sells lots and the one families will screenshot and send to relatives abroad. Give it a share action that captures the GL view to a PNG.

6.4 Performance budget

2,000+ lots. A naive scene with one mesh per lot is 2,000+ draw calls and will not hold frame rate on the mid-range Android hardware most of the customer base carries.

CONSTRAINT	TARGET	TECHNIQUE
Frame rate	60fps flagship, ≥30fps mid-range Android	`THREE.InstancedMesh` — one instanced draw per (lot type × status), per-instance transform and colour
Draw calls	< 60 at any camera position	Instancing + merged ground geometry + sprite label atlas
Lots resident	Viewport + one section margin	`lots:listInBbox` (already indexed on `geometry.bbox*`) driven by the camera frustum
Cold start to first frame	< 2.5 s	Progressive load: ground → section masses → lot instances → labels
Memory	< 250 MB	No per-lot textures; shared materials; dispose on unmount
Bundle cost	three.js tree-shaken; no full examples import	Import `OrbitControls` equivalents individually

Level of detail: beyond a distance threshold, a section renders as a single extruded mass with a count badge rather than its constituent lots. Families never see the seam; the GPU does.

6.5 Data and fidelity

The scene is driven by real data, not art: `lots.geometry.polygon` gives the footprint, `lots.dimensions` gives width and depth, `lots.type` selects the monument mass, `lots.status` selects the material. Lots with `geometryStatus === "placeholder"` are rendered in a visibly provisional style (translucent, hatched) — showing a surveyed-looking lot that has not been surveyed is a promise the park cannot keep.

Terrain: v1 uses a flat ground plane with the park's satellite orthophoto as a texture. Real elevation is Overflow, and needs a survey deliverable that does not exist yet.

6.6 Acceptance criteria

- **AC1** The scene renders live lot data — a lot's status changed in the web app is reflected in the app without a reload.
- **AC2** ≥30fps sustained on a mid-range Android reference device with a full section visible.
- **AC3** Tapping a lot selects it and opens its detail; the hit target tolerates a fat finger.
- **AC4** Placeholder-geometry lots are visually distinct from surveyed ones.
- **AC5** The scene degrades gracefully with no network: last-cached lots render, with a stale-data notice.
- **AC6** GL context loss (backgrounding the app) recovers without a crash.
- **AC7** A 3D-free fallback list view exists for devices where GL initialisation fails.

7. What I suggest adding

You asked what else belongs in this app. These are ranked by value to *this* business — a Philippine memorial park with a real Undas peak, families abroad, and a pre-need sales motion — not by what is fashionable in app development.

7.1 In scope for the 8 weeks

A. Push notifications — *2 days, very high value.* The single highest-leverage addition. The `reminderConfig` / `reminderDeliveries` tables and the cron scan already exist; push becomes a fourth channel next to email and SMS. Installment due in 7 days. Ceremony confirmed. Death anniversary approaching. Undas reminders. It is also the only mechanism that makes anyone open the app twice. Requires a **NEW** `pushTokens` table and an Expo Push action.

B. Find-a-Grave with walking directions — *3 days, very high value.* `convex/search.ts:172` `findGrave` already resolves a name to a lot; `lots.geometry.centroid` already holds the coordinates. Add device GPS and a walking bearing, and a visitor who has never been to the park can find their grandmother's grave without asking anyone. On Undas, this is the difference between a calm park and a queue at the office window. Works for visitors who are not customers — which makes it the app's best acquisition surface.

C. Digital tribute page per occupant — *3 days, high value.* Each `occupants` row gets a simple memorial page: name, dates, a photo, and short tributes from family. The `plaqueDrafts` table already exists, so the plaque and the page can share copy. Pair it with a QR code etched on the marker: a visitor scans it and reads the life, not just the dates. This is emotionally the right feature for a memorial park and it is what families share with relatives overseas. *Moderation matters* — tributes are user content; v1 restricts posting to the lot's contract owners and secondary estate owners.

D. Document vault — *2 days, high value.* `customerDocuments` and the contract / receipt PDF generators already exist. Surface them: contract PDF, every BIR receipt, the death certificate, the transfer deed — in one place, downloadable, offline-cached. Families lose these papers. The park has them.

E. Offline mode — *3 days, high value, non-negotiable in practice.* Signal at the park is unreliable. Cache the family's own lots, their receipts, the section map, and the last 3D scene state. ADR-0009 already sets an offline cache strategy for the web app; the app follows it.

F. Consultant contact and callback request — *1 day, high value for sales.* A family looking at a lot in 3D at 10pm should be able to ask a question. A callback request creates a `followUpActions` row — the table exists and staff already work that queue. This is the app's revenue path: browsing becomes a lead.

G. Family sharing — *2 days, medium-high value.* `familyEstates` supports secondary owners. Let them into the app: read-only access to the estate, its ceremonies, and its documents. Filipino estate purchases are household decisions made across several households, often across time zones.

7.2 Strongly recommended, but Overflow (weeks 9–14)

H. Undas mode (All Saints' / All Souls', Nov 1–2). The park's two busiest days by an order of magnitude. A seasonal mode that opens in October: visiting-window reservation to spread arrivals, parking guidance, a live crowd indicator, offline maps pre-downloaded, candle and flower pre-orders for pickup at the gate, and a "share your location with family" pin so relatives can find each other in a crowded park. This is the most *specifically valuable* feature in this whole document — it is Overflow only because it is seasonal and cannot ship half-built into a November deadline this year. **Plan it now for Undas 2027.**

I. Grave care services. Cleaning, repainting, flower placement, candle lighting on anniversaries — booked in-app, completed by groundskeepers, with a photo as proof of service. The `lotConditionLogs` and `perpetualCare` tables already exist. This is recurring revenue from families abroad who cannot visit, and it is the clearest post-sale monetisation the park has. Needs the staff field app to exist first, which is why it is Overflow.

J. Death anniversary reminders and memorial booking. `ceremonies.kind` already includes `memorial_anniversary` and `occupants` carries the dates. A gentle notification two weeks before, offering to arrange a memorial service or a candle lighting. Reverent, not promotional — the copy for this needs the client's blessing.

K. Agent / referral attribution. Commission tracking is an unresolved question in the original brief (§10). If the park sells through agents, in-app attribution of a reservation to a referring agent is straightforward once that policy exists.

L. QR at the gate. A code at the entrance and on each section marker that deep-links into the app at that location. Cheap, and it makes the physical park and the app one system.

7.3 Considered and not recommended for now

IDEA	WHY NOT
AR grave finding (camera overlay)	Impressive in a demo, fragile in sunlight, on slopes, and on mid-range hardware. Revisit after B ships and is measured.
In-app chat with staff	The park does not have staffing to answer a chat queue. A callback request (F) matches how the office actually works.
Social feed / community	Wrong register for a memorial park. Tributes (C) give the emotional value without the moderation liability.
Selling lots fully self-serve (no consultant)	Contradicts how this business closes, and multiplies the BIR and contract-compliance surface. Reservation (§4) is the right amount of self-serve.
Cryptocurrency / instalment BNPL	No.

8. Backend work required

All new work is Convex, in the existing deployment. Every function below follows the repo's conventions from §2.3.

8.1 New tables

`lotReservations` — the missing hold record.

```
lotReservations: defineTable({
  lotId: v.id("lots"),
  customerId: v.id("customers"),
  status: v.union(
    v.literal("pending_payment"),
    v.literal("held"),
    v.literal("converted"),
    v.literal("expired"),
    v.literal("cancelled"),
    v.literal("abandoned"),
  ),
  reservationFeeCents: v.number(),      // integer centavos, ADR-0007
  paymentIntentId: v.optional(v.id("paymentIntents")),
  createdAt: v.number(),
  paidAt: v.optional(v.number()),
  holdExpiresAt: v.optional(v.number()), // set on payment confirmation
  intentExpiresAt: v.number(),           // createdAt + 15 min
  convertedContractId: v.optional(v.id("contracts")),
  cancelledAt: v.optional(v.number()),
  cancellationReason: v.optional(v.string()),
  extendedByUserId: v.optional(v.id("users")),
  extensionReason: v.optional(v.string()),
  refundStatus: v.optional(v.union(
    v.literal("not_applicable"),
    v.literal("pending"),
    v.literal("refunded"),
    v.literal("forfeited"),
  )),
  source: v.union(v.literal("mobile"), v.literal("web"), v.literal("office")),
})
  .index("by_lot_status", ["lotId", "status"])
  .index("by_customer_status", ["customerId", "status"])
  .index("by_status_holdExpiresAt", ["status", "holdExpiresAt"])
  .index("by_status_intentExpiresAt", ["status", "intentExpiresAt"])
  .index("by_paymentIntent", ["paymentIntentId"]),
```

`ceremonyRequests` — the customer-side request that precedes a `ceremonies` row.

```
ceremonyRequests: defineTable({
  customerId: v.id("customers"),
  contractId: v.id("contracts"),
  lotId: v.id("lots"),
  kind: v.union(
    v.literal("interment"),
    v.literal("consecration"),
    v.literal("memorial_anniversary"),
  ),
  requestedAt: v.number(),
  requestedSlotStart: v.number(),
  requestedDurationMinutes: v.number(),
  chapelRequested: v.boolean(),
  pathwayRequested: v.boolean(),
  deceasedName: v.optional(v.string()),   // PII — ADR-0007 encryption applies
  notes: v.optional(v.string()),          // ≤ 500 chars, matching CEREMONY_NOTES_MAX_LENGTH
  status: v.union(
    v.literal("pending_review"),
    v.literal("confirmed"),
    v.literal("adjusted"),
    v.literal("declined"),
    v.literal("cancelled_by_customer"),
  ),
  reviewedByUserId: v.optional(v.id("users")),
  reviewedAt: v.optional(v.number()),
  ceremonyId: v.optional(v.id("ceremonies")),  // set on confirmation
  staffMessage: v.optional(v.string()),
})
  .index("by_status_requestedAt", ["status", "requestedAt"])
  .index("by_customer", ["customerId"])
  .index("by_lot", ["lotId"]),
```

`pushTokens` — device registration for Expo Push.

```
pushTokens: defineTable({
  userId: v.id("users"),
  expoPushToken: v.string(),
  platform: v.union(v.literal("ios"), v.literal("android")),
  deviceName: v.optional(v.string()),
  registeredAt: v.number(),
  lastSeenAt: v.number(),
  revokedAt: v.optional(v.number()),
})
  .index("by_user_active", ["userId", "revokedAt"])
  .index("by_token", ["expoPushToken"]),
```

`tributes` *(feature C)* — occupant memorial content, moderated by ownership.

8.2 New functions

MODULE	FUNCTION	ROLE GATE	NOTES
`convex/reservations.ts`	`listReservableLots`	`customer`	Filtered, paginated, geometry-light
	`createReservationIntent`	`customer`	Validates eligibility + concurrent-hold cap; creates `paymentIntents` row; returns checkout URL
	`confirmReservationFromWebhook`	*internal*	The serializable hold transaction (§4.4)
	`cancelMyReservation`	`customer`	Own reservation only
	`listMyReservations`	`customer`	Scoped server-side
	`listActiveHolds`	`admin`, `office_staff`	Staff queue in the web app
	`extendHold`	`admin`, `office_staff`	Once, +7 days, reason required
	`convertHoldToContract`	`admin`, `office_staff`	Links the reservation to the new contract, credits the fee
`convex/ceremonyRequests.ts`	`getAvailableSlots`	`customer`	Advisory availability; respects blackouts + lead time
	`requestCeremony`	`customer`	Ownership-scoped; creates `pending_review`
	`cancelMyCeremonyRequest`	`customer`	Only while `pending_review`
	`listMyCeremonyRequests`	`customer`	
	`listPendingRequests`	`admin`, `office_staff`	Staff queue
	`confirmCeremonyRequest`	`admin`, `office_staff`	Calls existing `scheduleCeremony`; handles `SCHEDULING_CONFLICT`
	`declineCeremonyRequest`	`admin`, `office_staff`	Reason required, surfaced to the family
`convex/pushTokens.ts`	`registerPushToken` / `revokePushToken`	`customer`	

MODULE	FUNCTION	ROLE GATE	NOTES
`convex/actions/sendPushNotifications.ts`	Expo Push dispatch	*internal*	`"use node"` — **must not** be imported by any V8 file (see §10 R4)
`convex/crons.ts`	`expireStaleReservationIntents`	—	Every 5 min; `pending_payment` past `intentExpiresAt`
	`expireElapsedHolds`	—	Every 15 min; `held` past `holdExpiresAt` → lot back to `available`
	`notifyExpiringHolds`	—	Daily; push at T-3 days and T-1 day

8.3 Changes to existing code

- `convex/schema.ts` — four new tables; add `"reservation"` and `"ceremony_request"` to the `auditLog.entityType` union.
- `convex/lib/lotStatus.ts` (state machine) — register the reservation-driven transitions so ADR-0006 stays the single source of truth for lot status.
- `convex/portal.ts` — extend the customer surface with reservation and request reads.
- `convex/http.ts` — the existing gateway webhook handlers must route reservation-fee intents to `confirmReservationFromWebhook` rather than the installment path. **This is a modification to a financial cornerstone; it needs a dedicated review.**
- Web app — two new staff screens: active holds, and the ceremony-request queue. Without these, the app's requests land nowhere.
- `src/lib/scene3d/` — extract the shared three.js module from `Phase3DMap.tsx` (§6.2).

9. Timeline — 8 weeks

Team assumed: 1 mobile developer (full-time), 1 backend/full-stack developer (full-time), designer at ~30%, QA at ~30% from week 4. A single developer doing all of this takes 13–15 weeks; that is not a judgement about the developer, it is arithmetic.

Week 0 — start these on day one, in parallel with everything

Administrative lead times are the most commonly underestimated risk in a two-month mobile project. None of this is engineering work and all of it blocks shipping:

- **Apple Developer Program enrolment as an organisation.** Requires a D-U-N-S number for Cases Land Inc.; obtaining one takes up to 5 business days, and Apple's review of the enrolment adds more. **This can consume two weeks of calendar time and blocks TestFlight entirely.**
- **Google Play Developer account** + the identity verification Google now requires.
- Payment gateway credentials for the app's return-deep-link redirect URLs.
- Privacy policy and terms, published at a public URL (both stores require it).
- App name, icon, splash, and store screenshots from the brand system.

Weeks 1–8

WEEK	MOBILE	BACKEND	MILESTONE
1	Expo scaffold, `expo-router`, brand token module, navigation shell, **auth spike** (§10 R1)	`lotReservations` + `pushTokens` schema; extract `src/lib/scene3d/`	App builds and runs on both platforms; a customer can log in
2	Browse + lot detail + "my estate" (contracts, payments, receipts — all existing queries)	`listReservableLots`; reservation eligibility rules	Read-only app is fully usable
3	**3D v1** — `expo-gl` scene, gesture camera, instanced lots, tap-select	Reservation mutations + the hold transaction (§4.4)	3D renders live data at ≥30fps on the reference Android device
4	Reservation flow UI: review → checkout → deep-link return → result	Webhook routing, expiry crons, staff holds screen	**Internal build 1** to TestFlight / Play Internal. Reservation works end-to-end in sandbox.
5	Booking flow UI: calendar, slots, request, status	`ceremonyRequests` + `getAvailableSlots` + confirm/decline; staff queue screen	A request can be made and confirmed end-to-end
6	Push registration + handling; Find-a-Grave with directions; document vault; callback request	Expo Push action; reminder-channel wiring; `tributes` if pace allows	**Feature complete.** Beta build to real staff and 5–10 friendly families.
7	Offline caching, 3D LOD tuning, accessibility, brand copy pass, error and empty states	Security review, audit-coverage check, load test on `listInBbox` and slot queries	**Store submission.** Data-safety and privacy forms filed.

WEEK	MOBILE	BACKEND	MILESTONE
8	Review feedback, bug fixes, resubmission, launch comms	Production gateway keys, monitoring, runbook update	**Launch** (subject to review outcome)

What overflows past week 8

Named honestly rather than pretended away: Undas mode (§7.2 H), grave care services (I), anniversary memorial booking (J), multi-lot family-estate reservations, tributes if week 6 runs tight, real terrain elevation in the 3D scene, and agent attribution. That is a coherent 4–6 week Phase 2.

Where the plan bends if it slips

In order — first to go, last to go:

1. Tributes (C) → Phase 2.
2. Family sharing (G) → Phase 2.
3. 3D reduces to two views (park overview + lot in situ), dropping the section view.
4. Booking ships iOS-first; Android follows two weeks later.
5. **Never cut:** reservation expiry crons, server-side ownership scoping, audit emission, or the refund path in §4.4. These are the ones that hurt real families and real money.

10. Risks

#	RISK	LIKELIHOOD	IMPACT	MITIGATION
R1	`@convex-dev/auth` on React Native needs more integration work than assumed (secure token storage, refresh on cold start, deep-link return after gateway checkout)	Medium	High — blocks everything	**Spike in week 1, before any feature work.** Fallback: a short-lived, single-use mobile session token minted by a Convex mutation from the existing web session.
R2	Apple organisation enrolment (D-U-N-S) blows through two weeks	**High**	High — no TestFlight, no submission	Start day one. If it stalls, ship Android first; Play review is days, not weeks.
R3	3D does not hold frame rate on mid-range Android	Medium	Medium	Instancing and LOD from the first commit, not as an optimisation pass. Buy a reference device (a ₱10–15k Android) in week 1 and test on it weekly.
R4	A `"use node"` action gets imported into a V8 query/mutation and the whole deploy fails	Medium	Medium	This has already happened once in this repo (`archivalExports` → `node:zlib`). **Rule: never import anything — even a string constant — from a `"use node"` file into a V8 file.** Put shared constants in `convex/lib/`.
R5	Reservation double-payment race produces a real refund incident	Low	High — trust	§4.4's transaction ordering, plus the automatic refund path, plus a staff alert on every `abandoned/lot_taken`. Load-test concurrency in week 7.
R6	Payment gateway rejects the mobile redirect flow or the deep-link return URL	Medium	High	Validate the full sandbox round-trip in week 4, not week 7.
R7	App Store rejects payments as requiring in-app purchase	Low	High	Cemetery lots and interment services are real-world goods and services consumed outside the app, which is the standard exemption. Document this in review notes pre-emptively.
R8	Staff never work the request queue, so families' bookings sit unanswered	Medium	High — worse than not shipping	The web-side queue screens are in scope (§8.3), plus an ageing alert on any `pending_review` older than one business day. Train staff in week 6, not week 8.
R9	Customer PII (deceased names, contact details) leaves the Philippines' Data Privacy Act envelope via push payloads or crash logs	Medium	High	Push payloads carry ids only, never names. Scrub PII from Sentry/crash reporting. Extend `docs/threat-model.md`.

#	RISK	LIKELIHOOD	IMPACT	MITIGATION
R10	Timeline is treated as a commitment to the client before week 4's evidence exists	Medium	Medium	Communicate the plan as *target*; re-forecast at the week 4 milestone with real velocity.

11. Non-functional requirements

Performance. Cold start to interactive < 3 s on the reference Android device. 3D first frame < 2.5 s. Any list screen renders in < 1 s on 4G. Reuse the discipline of ADR-0016's performance budget gates; add a mobile equivalent to CI.

Offline. The family's own lots, contracts, receipts, and the section map remain readable with no connection, with an explicit "last updated" stamp. Writes queue and replay; a reservation is **never** written optimistically offline — it needs a live transaction (§4.4).

Security. Every function `requireRole`-gated. Ownership derived server-side, never client-supplied. Tokens in `expo-secure-store` (Keychain / Keystore), never AsyncStorage. Certificate pinning on the Convex connection is Overflow. Jailbreak/root detection is not in scope.

Privacy (RA 10173, Data Privacy Act). Explicit consent at onboarding for the data the app collects. Location used only in the foreground, only for wayfinding, with a clear purpose string. Data-subject export and deletion route through the existing `convex/dataSubject.ts`. Both stores' data-safety declarations must match reality exactly.

Accessibility. WCAG 2.1 AA equivalents: minimum 44×44pt touch targets, dynamic type support up to 200%, screen-reader labels on every control, 4.5:1 contrast. The emerald `#1D5C4D` on ivory `#F6F2EA` pairing passes; gold `#C9A96B` is an accent only and must never carry text meaning. Grief impairs attention — clear beats clever everywhere.

Localisation. English at launch. Tagalog strings extracted from day one (all copy through a `t()` function, no inline literals) so a Tagalog build is a translation pass, not a refactor. Ilocano is worth considering for La Union specifically — ask the client.

Brand. Cormorant Garamond for ceremonial headings, Manrope for operational text, JetBrains Mono for lot codes. Gold rationed to hairline rules and the mark inlay, never as a fill. Letter and notification sign-off: "With reverence, / The Estate Office / APOSTLE PAUL MEMORIAL PARK".

Observability. Crash reporting with PII scrubbed. Funnel instrumentation on the two flows that matter: browse → reserve → paid, and request → confirmed. Convex function latency and error rates on the new mobile-facing functions.

---

12. Open questions — needed before the relevant week

#	QUESTION	BLOCKS	NEEDED BY
Q1	Reservation fee: how much, and what happens to it on expiry or cancellation — refundable, partly refundable, or forfeit?	§4 entirely	**Week 2**
Q2	Minimum lead time for an interment request, and the park's committed response time for confirming one	§5	Week 4
Q3	Hold duration — is 14 days right for how this park actually sells?	§4	Week 2
Q4	Blackout dates: Nov 1–2 confirmed? Holy Week? Any weekly closure?	§5	Week 4
Q5	Who at the office owns the request queue, and what is their working-hours commitment?	R8	Week 5
Q6	Does a reservation fee require a BIR receipt at the moment of payment, or only on conversion to a contract? *(This is an unresolved question from the original brief §10 and has real compliance weight.)*	§4, §8.3	**Week 2**
Q7	Are tributes (§7.1 C) acceptable to the client, and who moderates them?	§7.1 C	Week 5
Q8	Tagalog at launch, or English-only? Ilocano?	§11	Week 5
Q9	Who owns the Apple and Google developer accounts — Cases Land Inc. or the agency? *(Transferring a published app between accounts later is painful.)*	Week 0	**Immediately**
Q10	Is the park's satellite orthophoto available at sufficient resolution for the 3D ground texture?	§6.5	Week 3

13. Definition of done

The app ships when all of the following are true:

- A family can browse, view a lot in 3D, reserve it with a real payment, and see the hold in their account — on both iOS and Android.
- A family can request an interment and receive a confirmation, and the office can confirm or adjust it from the web app.
- Holds expire automatically and return lots to inventory, verified in production.
- Every customer-facing mutation is role-gated, ownership-scoped, and audited.
- Offline reading works for owned lots, contracts, and receipts.
- Both stores have approved the build, and the data-safety declarations match what the app actually does.
- `npm run typecheck`, `npm run lint`, `npm test`, and the mobile E2E suite are green in CI.
- `docs/runbook.md` covers mobile release, rollback, and push-notification incident response.

Prepared for Cases Land Inc. · Apostle Paul Memorial Park. With reverence, / The Estate Office

PART TWO

Mobile Companion App — Epic Breakdown

The same scope, broken into epics and stories with acceptance criteria.

Decomposes mobile-app-spec.md into implementable stories. Story numbering is `M<epic>.<story>` so it never collides with the existing web backlog in epics.md.

Read the spec first. This document assumes its §2 inventory (what the backend already provides), §4–§6 feature rules, and §8 backend surface. Stories here carry acceptance criteria and sequencing; they do not restate the rules.

Requirements inventory

Numbered `MFR*` (mobile functional) and `MNFR*` (mobile non-functional) to stay distinct from the web app's `FR*` series.

Functional

1. Identity

- **MFR1:** A customer can authenticate in the app with the credentials they already use for the web portal. [W1]
- **MFR2:** The app persists the session across cold starts in device secure storage. [W1]
- **MFR3:** A customer can redeem a portal invite link on a device and land signed in. [W2]
- **MFR4:** A customer can sign out, which revokes the device's push token. [W6]

2. Browse & 3D

- **MFR5:** A customer can browse available lots filtered by section, type, price band, and status. [W2]
- **MFR6:** A customer can view a lot's detail — section copy, dimensions, price, status. [W2]
- **MFR7:** A customer can explore the park as a 3D scene driven by live lot data. [W3]
- **MFR8:** A customer can view a single lot in situ among its neighbours and share that view as an image. [W3]
- **MFR9:** Lots with placeholder geometry are visually distinguished from surveyed lots. [W3]
- **MFR10:** The app falls back to a list view when GL initialisation fails. [W3]

3. Reservation

- **MFR11:** A customer can place a deposit-backed, time-boxed hold on an available lot. [W3–4, gated on Q1, Q3, Q6]
- **MFR12:** A hold is created only after the payment gateway confirms the reservation fee. [W4]
- **MFR13:** A customer can view and cancel their own active holds. [W4]
- **MFR14:** A hold past its expiry is released automatically and the lot returns to inventory. [W4]
- **MFR15:** Office staff can see every active hold, extend one, and convert one to a contract. [W4]
- **MFR16:** A reservation fee paid against a lot that was taken first is refunded automatically. [W4]

4. Booking

- **MFR17:** A customer can see available ceremony slots for a chosen date on a lot they own. [W5, gated on Q2, Q4]
- **MFR18:** A customer can request an interment, consecration, or memorial anniversary. [W5]
- **MFR19:** A customer can cancel their own request while it is pending review. [W5]
- **MFR20:** Office staff can confirm, adjust, or decline a request from the web app. [W5]
- **MFR21:** Confirmation routes through the existing `scheduleCeremony` guard. [W5]

5. Engagement & retrieval

- **MFR22:** A customer receives push notifications for due installments, hold expiry, and ceremony outcomes. [W6]
- **MFR23:** Any visitor can look up a grave by name and receive walking guidance to it. [W6]
- **MFR24:** A customer can read and download their contracts, receipts, and uploaded documents. [W6]
- **MFR25:** A customer can request a callback from a consultant. [W6]
- **MFR26:** A secondary estate owner can read the estate, its ceremonies, and its documents. [W6]

- **MFR27:** A contract owner can publish a tribute to an occupant. [Overflow]

Non-functional

- **MNFR1:** Cold start to interactive under 3 s on the reference mid-range Android device.
- **MNFR2:** 3D holds ≥30fps on the reference device with a full section visible; <60 draw calls.
- **MNFR3:** Owned lots, contracts, and receipts remain readable offline with a "last updated" stamp.
- **MNFR4:** Every mobile-facing Convex function is role-gated, ownership-scoped server-side, and audited.
- **MNFR5:** Session tokens live in Keychain / Keystore, never AsyncStorage.
- **MNFR6:** Push payloads carry ids only — never names, amounts, or contact details.
- **MNFR7:** Touch targets ≥44×44pt; dynamic type to 200%; 4.5:1 contrast; screen-reader labels on every control.
- **MNFR8:** All copy passes through `t()` with no inline string literals.
- **MNFR9:** Both stores' data-safety declarations match the app's actual collection.

Epic list

EPIC	TITLE	WEEKS	DEPENDS ON
M0	Release prerequisites (non-engineering)	W0, runs throughout	—
M1	Mobile foundation & customer identity	W1–W2	M0.1 for device builds
M2	Browse & 3D displays	W2–W3	M1
M3	Online reservation	W3–W4	M1, M2.2, Q1/Q3/Q6 answered
M4	Online booking	W5	M1, Q2/Q4 answered
M5	Engagement & retrieval	W6	M1
M6	Hardening & release	W7–W8	all above

Critical path: M1.2 (auth spike) → M1.4 (app shell) → M2.2 (scene extraction) → M3.3 (hold transaction) → M6.5 (store submission). Everything else has slack.

Gates. M3 cannot start its payment stories until Q1 (fee refund policy) and Q6 (BIR receipt timing) are answered. M4 cannot ship its slot rules until Q2 (lead time) and Q4 (blackout dates) are answered. Both gates are named in the spec §12 with the weeks they bite.

Epic Mo: Release prerequisites [Wo]

Not engineering work, but it blocks shipping and has the longest lead time in the project. Tracked as stories so it has an owner and a due date.

Story Mo.1: Apple and Google developer accounts exist

As the project owner, I want organisation accounts on both app stores, So that the team can distribute test builds from week 4 and submit in week 7.

Acceptance Criteria:

Given Cases Land Inc. has no D-U-N-S number, **When** the owner requests one from Dun & Bradstreet, **Then** the number is obtained and recorded before Apple Developer Program enrolment is submitted.

Given enrolment is submitted, **When** Apple completes its review, **Then** the team can create an App Store Connect record and push a TestFlight build.

Given the accounts are created, **When** ownership is recorded, **Then** it is explicit whether Cases Land Inc. or the agency holds them (spec Q9) — transferring a published app later is painful.

Blocks: every device-distributed build. *Risk:* spec R2. *Start day one.*

Story Mo.2: Privacy policy and terms are published

Given both stores require a public privacy policy URL, **When** the policy is drafted against RA 10173 and what the app actually collects, **Then** it is published at a stable URL and linked from the app's account screen.

Given the app collects foreground location for wayfinding, **When** the policy is written, **Then** it names that purpose specifically rather than in a general clause.

Story Mo.3: Gateway sandbox credentials cover the mobile redirect

Given the app returns from hosted checkout via a deep link, **When** the redirect URL scheme is registered with GCash, Maya, and the card provider, **Then** a sandbox payment completes the full round-trip from app to gateway to webhook to app.

Do this in week 4, not week 7 — spec R6.

Story Mo.4: Store listing assets exist

Given the brand system defines logo, palette, and typography, **When** the icon, splash, and screenshots are produced, **Then** they honour the four voice pillars and contain no urgency or promotional language.

Epic M1: Mobile foundation & customer identity [W1–W2]

Expo scaffold, the auth spike that de-risks everything downstream, the brand token module, navigation, and the read-only "my estate" surface built entirely on Convex functions that already exist.

Story M1.1: Expo project boots on both platforms

As a developer, I want a running Expo app wired to the existing Convex deployment, So that feature work has somewhere to land.

Acceptance Criteria:

Given a clean clone, **When** the developer runs the documented setup, **Then** the app builds and runs on an iOS simulator and an Android emulator in under 10 minutes.

Given the app is running, **When** it connects to `beaming-boar-935`, **Then** a smoke query resolves and the connection state is visible in a debug screen.

Given `convex/_generated` is not committed, **When** the app calls a Convex function, **Then** it uses `makeFunctionReference` exactly as the web client does — no codegen step is introduced.

Given the repo's TypeScript settings, **When** the mobile `tsconfig` is written, **Then** `noUncheckedIndexedAccess` and strict mode are on, matching the web app.

Story M1.2: Customer authentication works on device — SPIKE FIRST

As a customer, I want to sign in with the credentials I already use for the portal, So that I do not need a second account.

Acceptance Criteria:

Given `@convex-dev/auth` has not been exercised on React Native in this project, **When** the spike runs at the start of week 1, **Then** it produces a written answer on token storage, refresh on cold start, and deep-link return after an external browser round-trip — **before** any feature work depends on it.

Given a customer with an existing portal account, **When** they sign in on the app, **Then** `getCurrentCustomer` resolves and their name appears on the account screen.

Given a signed-in customer, **When** they force-quit and relaunch, **Then** they remain signed in, with the token read from `expo-secure-store` (MNFR5).

Given the spike finds the library path unworkable, **When** the fallback is adopted, **Then** a short-lived single-use mobile session token is minted by a Convex mutation from the existing web session, and the decision is recorded as an ADR.

Given a user whose role is not `customer`, **When** they sign in, **Then** they are told the app is for families and staff should use the web app — not shown an empty shell.

Risk: spec R1. *This is the single highest-leverage story in the backlog.*

Story M1.3: Brand tokens are available to the app

Given `tailwind.config.ts` is the single token source, **When** the token module is generated, **Then** the app consumes emerald, forest, moss, ivory, stone, gold, and ink from that derived module — no hand-copied hex values.

Given the brand rations gold to hairline rules and the mark, **When** a component uses gold as a fill, **Then** review rejects it.

Given the type system, **When** headings, body, and lot codes render, **Then** they use Cormorant Garamond, Manrope, and JetBrains Mono respectively, with real fallback stacks.

Story M1.4: App shell, navigation, and copy pipeline

Given the app has four top-level destinations (Explore, My Estate, Find a Grave, Account), **When** the shell is built with `expo-router`, **Then** each tab is reachable, labelled for screen readers, and has a ≥44×44pt target (MNFR7).

Given MNFR8, **When** any string renders, **Then** it comes from `t()`; a lint rule fails the build on inline user-facing literals.

Given a screen has no data, **When** it renders, **Then** it shows a written empty state in the park's voice — never a spinner that never resolves.

Story M1.5: Portal invite redemption on device

Given a customer receives a portal invite email, **When** they open the link on a phone with the app installed, **Then** the deep link opens the app at the accept-invite screen and `acceptPortalInvite` completes.

Given the app is not installed, **When** they open the link, **Then** they land on the existing web accept-invite page — the web flow is not broken by adding the deep link.

Given an expired or already-used token, **When** it is opened, **Then** the app explains what happened and offers to contact the office.

Story M1.6: "My estate" read surface

As a customer, I want to see my contracts, payments, and receipts, So that the app is useful from the first release even before reservation ships.

Acceptance Criteria:

Given the existing `listCustomerContracts`, `getCustomerContractDetail`, `listCustomerPayments`, and `listCustomerReceipts` queries, **When** the screens are built, **Then** no new backend function is written for this story.

Given money values arrive as integer centavos, **When** they render, **Then** they are formatted with a port of `formatPeso` — rounding is not reimplemented.

Given an installment contract, **When** the detail screen renders, **Then** it shows the next due date and remaining installments; a full-payment contract shows neither.

Given a crafted contract id belonging to another customer, **When** the app requests it, **Then** the handler returns `null` and the screen shows a not-found state — existence does not leak.

Story M1.7: In-app installment payment

Given `createGatewayPaymentIntent` already exists, **When** a customer pays an installment from the app, **Then** the hosted checkout opens via `expo-web-browser` and the result returns through the deep link.

Given a payment succeeds, **When** the webhook lands, **Then** the contract balance updates reactively on the open screen without a manual refresh.

Given the customer backgrounds the app mid-checkout, **When** they return, **Then** the intent's live status resolves the screen — a stale "pending" never sticks.

Epic M2: Browse & 3D displays [W2–W3]

The features that justify a native app. Story M2.2 is a refactor of working web code and must land before the mobile scene starts.

Story M2.1: Browse and filter reservable lots

Given a customer opens Explore, **When** the list loads, **Then** it shows available lots filtered by section, type (`single` / `family` / `mausoleum` / `niche`), price band, and status.

Given the park has 2,000+ lots, **When** the list renders, **Then** results are paginated and the payload omits full polygon arrays — the list does not need geometry.

Given a lot's status changes in the web app, **When** the customer is looking at the list, **Then** it updates reactively.

Story M2.2: Extract the shared 3D scene module

As a developer, I want scene construction to live in one platform-neutral module, So that web and mobile render the same park from the same code.

Acceptance Criteria:

Given `Phase3DMap.tsx` currently holds scene construction inside a mount effect, **When** the extraction lands, **Then** `src/lib/scene3d/` exports `buildParcelScene`, `statusMaterials`, and `pickLotAtPointer` with no DOM or React Native imports.

Given the extraction, **When** the web `/phase-3d` route is loaded, **Then** it renders identically to before — verified by the existing checks plus a visual pass.

Given the module is platform-neutral, **When** it is imported from the Expo app, **Then** it compiles without a bundler shim.

Blocks: M2.3, M2.4, M2.5.

Story M2.3: 3D park scene renders on device

Given the shared scene module, **When** the mobile shell mounts it in an `expo-gl` view, **Then** the park renders from live `lots` data with status colouring (MFR7).

Given 2,000+ lots, **When** the scene builds, **Then** lots are drawn with `THREE.InstancedMesh` — one instanced draw per type × status — and total draw calls stay under 60 (MNFR2).

Given the reference mid-range Android device, **When** a full section is in view, **Then** the scene sustains ≥30fps.

Given the camera moves, **When** new lots enter the frustum, **Then** they load via `listInBbox` rather than a full-park fetch.

Given the app is backgrounded and resumed, **When** the GL context is lost and restored, **Then** the scene rebuilds without a crash.

Story M2.4: Touch camera and lot selection

Given OrbitControls binds DOM events and cannot be used, **When** the gesture handler is wired, **Then** one finger orbits, two fingers pan and pinch-zoom, and a double-tap frames a lot.

Given a customer taps a lot, **When** the raycast resolves, **Then** the lot is selected, highlighted, and its detail is reachable — with a hit target that tolerates a fat finger.

Given section labels cannot be DOM nodes, **When** they render, **Then** they are `THREE.Sprite` textures that occlude correctly.

Story M2.5: Lot in situ, and sharing it

Given a lot detail screen, **When** the in-situ view opens, **Then** the camera frames that lot among its neighbours with the monument mass shown for its type (MFR8).

Given a customer wants to show family abroad, **When** they tap share, **Then** the GL view is captured to a PNG and handed to the system share sheet.

Given a lot with `geometryStatus === "placeholder"`, **When** it renders, **Then** it is visibly provisional — translucent and hatched — because showing an unsurveyed lot as surveyed is a promise the park cannot keep (MFR9).

Story M2.6: Graceful degradation without GL

Given a device where GL initialisation fails, **When** the customer opens Explore, **Then** a list view renders with the same filters and no error dialog (MFR10).

Given no network, **When** the scene opens, **Then** the last cached lots render with a "last updated" stamp rather than an empty scene.

Epic M3: Online reservation [W3–W4]

The largest new backend surface, and the one where correctness is money. **Gated on Q1 and Q6** — do not build the fee stories until the refund policy and the BIR receipt timing are answered.

Story M3.1: `lotReservations` schema and state machine registration

Given spec §8.1, **When** the table is added, **Then** it carries the six-literal status union, both expiry timestamps, the payment-intent FK, the refund status, and all five indexes.

Given ADR-0006 makes the state machine the single source of truth for lot status, **When** reservation-driven transitions are added, **Then** they are registered there — no raw `db.patch` on `lots.status` anywhere in the reservation path.

Given ADR-0004, **When** the `auditLog.entityType` union is extended, **Then** it gains `"reservation"` so per-reservation `by_entity` lookups work.

Story M3.2: Customer creates a reservation intent

Given an available, non-retired lot, **When** a customer requests a reservation, **Then** a `pending_payment` row is created with a 15-minute `intentExpiresAt` and a gateway checkout URL is returned.

Given a customer already holding three active reservations, **When** they request a fourth, **Then** the mutation refuses server-side with a message that explains the cap.

Given a lot that is reserved, sold, occupied, or retired, **When** a reservation is requested, **Then** it is refused server-side — not merely hidden in the UI.

Given the lot is not held yet at this point, **When** another customer views it, **Then** it still shows as available — the hold happens at confirmation, not here.

Story M3.3: The hold transaction

As the system, I want a lot held exactly once, So that two families paying at the same moment cannot both own the same grave.

Acceptance Criteria:

Given a gateway webhook confirming a reservation fee, **When** `confirmReservationFromWebhook` runs, **Then** it re-reads the lot inside the mutation, checks the status, transitions to `reserved`, sets `holdExpiresAt`, and emits an audit row — all in one serializable transaction.

Given two confirmations for the same lot arrive concurrently, **When** both mutations run, **Then** exactly one produces a `held` reservation and the other produces `abandoned` with `failureReason: "lot_taken"`.

Given a reservation is `abandoned` for `lot_taken`, **When** the mutation completes, **Then** a refund action is scheduled immediately and a staff alert is raised.

Given the customer whose payment lost the race, **When** they open the app, **Then** they see a notice in the park's voice: the lot was taken moments before their payment completed, and their fee is being returned in full.

Risk: spec R5. *Load-test this in week 7.*

Story M3.4: Webhook routing distinguishes reservation fees from installments

Given `convex/http.ts` currently routes every confirmed intent to the installment path, **When** reservation intents are introduced, **Then** the handler dispatches on intent kind and reservation fees never post as installment payments.

Given this modifies a financial cornerstone, **When** the change is proposed, **Then** it gets a dedicated review before merge.

Given a webhook arrives twice for the same intent, **When** it is processed, **Then** the second is a no-op — exactly-once semantics hold.

Story M3.5: Holds expire on their own

Given a `pending_payment` row past `intentExpiresAt`, **When** the 5-minute cron runs, **Then** it is marked `abandoned` and no lot status changed (none was held).

Given a `held` reservation past `holdExpiresAt`, **When** the 15-minute cron runs, **Then** the lot returns to `available` through the state machine, an audit row is emitted, and the customer is notified (MFR14).

Given a hold approaching expiry, **When** the daily cron runs, **Then** the customer is notified at T-3 days and T-1 day.

This story is on the never-cut list. A hold that cannot expire silently destroys inventory.

Story M3.6: Customer views and cancels their holds

Given a customer with active holds, **When** they open My Estate, **Then** each hold shows the lot, the days remaining, and what happens next.

Given a customer cancels their own hold, **When** the mutation runs, **Then** the lot returns to `available` immediately and the refund status follows the Q1 policy.

Given a hold belonging to another customer, **When** its id is requested, **Then** the handler returns `null`.

Story M3.7: Reservation flow UI

Given a customer on a lot detail screen, **When** they choose to reserve, **Then** the review step states the fee, the hold duration, and the refund policy in plain language, with an explicit consent checkbox.

Given the review screen, **When** copy is reviewed, **Then** it contains no scarcity language, no countdown styled as pressure, and no exclamation marks.

Given the checkout sheet is open, **When** another customer's hold lands on the same lot, **Then** the live status updates in front of the customer before they pay.

Given the payment fails or is abandoned, **When** the customer returns, **Then** the result screen explains plainly and offers to try again.

Story M3.8: Staff holds queue on the web app

Given office staff open the new holds screen, **When** it loads, **Then** every active hold is listed with customer, lot, expiry, and source.

Given a family needs more time, **When** staff extend a hold, **Then** the extension is once only, +7 days, with a required reason, and it is audited.

Given a family completes a purchase, **When** staff convert the hold, **Then** a contract is created, the reservation becomes `converted`, and the fee is credited to the first payment.

Without this screen the app's holds land nowhere. It ships in the same week as M3.7.

Epic M4: Online booking [W5]

Gated on Q2 and Q4. Customers request; staff confirm. The authoritative conflict check stays where it already is.

Story M4.1: `ceremonyRequests` schema

Given spec §8.1, **When** the table is added, **Then** it carries the five-literal status union, the requested slot, chapel and pathway flags, the optional deceased name as encrypted PII (ADR-0007), and the resulting `ceremonyId`.

Given ADR-0004, **When** the audit union is extended, **Then** it gains `"ceremony_request"`.

Story M4.2: Available slots query

Given a customer picks a date, **When** `getAvailableSlots` runs, **Then** it returns open windows accounting for existing ceremonies, chapel and pathway occupancy, park operating hours, blackout dates, and the lead-time floor.

Given the query is advisory only, **When** its logic is written, **Then** it calls the same primitives as `assertNoBookingConflict` rather than reimplementing overlap rules — the authoritative check must not drift from the advisory one.

Given Nov 1–2 and any park-defined closure, **When** they are configured in `appSettings`, **Then** no slot is offered on those dates.

Story M4.3: Customer requests a ceremony

Given a customer with an active contract on a lot, **When** they submit a request, **Then** a `pending_review` row is created, staff are notified, and **no `ceremonies` row is written**.

Given a customer without ownership of the lot, **When** they attempt a request, **Then** it is refused server-side.

Given a secondary owner on the lot's family estate, **When** they submit a request, **Then** it is accepted.

Given a request inside the lead-time floor or on a blackout date, **When** it is submitted, **Then** it is refused server-side with a plain explanation.

Given notes exceed the ceremony notes limit, **When** the form is submitted, **Then** validation refuses it with the same bound the web app enforces.

Story M4.4: Customer tracks and cancels a request

Given a pending request, **When** the customer opens it, **Then** they see its status and the stated response commitment — "The Estate Office will confirm within one business day".

Given a pending request, **When** the customer cancels it, **Then** it moves to `cancelled_by_customer` and staff are notified.

Given a confirmed ceremony, **When** the customer opens it, **Then** cancellation is not offered in-app; they are directed to contact the office.

Story M4.5: Staff confirm, adjust, or decline

Given the request queue on the web app, **When** staff confirm a request, **Then** `scheduleCeremony` is called and its `assertNoBookingConflict` guard runs as the authoritative check (MFR21).

Given confirmation raises `SCHEDULING_CONFLICT`, **When** the error is handled, **Then** the request returns to the family with a plain-language explanation and alternative slots — never a raw error code.

Given staff adjust the slot, **When** the family is notified, **Then** the notice states what changed and why.

Given staff decline, **When** they submit, **Then** a reason is required and it is surfaced to the family.

Given a request pending longer than one business day, **When** the ageing check runs, **Then** staff are alerted — an unanswered booking queue is worse than not shipping the feature (spec R8).

Story M4.6: Confirmed ceremony detail

Given a confirmed ceremony, **When** the customer opens it, **Then** they see date, time, section wayfinding, add-ons, and an add-to-device-calendar action.

Epic M5: Engagement & retrieval [W6]

Cheap features with high value, because the backend for most of them already exists.

Story M5.1: Push token registration

Given a signed-in customer, **When** they grant notification permission, **Then** an Expo push token is stored against their user with platform and device name.

Given a customer signs out, **When** the session ends, **Then** the token is revoked (MFR4).

Given MNFR6, **When** any push payload is constructed, **Then** it carries ids only — never names, amounts, or contact details.

Story M5.2: Push becomes a fourth reminder channel

Given the existing reminder cron and `reminderDeliveries`, **When** push is added, **Then** it slots in beside email and SMS rather than becoming a parallel scan.

Given an installment due in 7 days, due today, or 3 days overdue, **When** the scan runs, **Then** the customer receives one push per rule offset — the existing exactly-once dedup applies.

Given a ceremony is confirmed, adjusted, or declined, **When** the mutation completes, **Then** the family is pushed.

Given a hold approaches expiry, **When** the daily cron runs, **Then** the customer is pushed at T-3 and T-1.

Story M5.3: Find a grave, with walking guidance

Given any visitor — signed in or not, **When** they search a name, **Then** `findGrave` resolves it to a lot and shows the section, block, and row.

Given the visitor grants foreground location, **When** they choose to walk there, **Then** the app shows a bearing and distance to the lot centroid, updating as they move.

Given location permission is denied, **When** the screen renders, **Then** it still shows the lot on the section map with written directions from the gate.

Given MNFR3, **When** the visitor has no signal at the park, **Then** the cached section map and the resolved lot still render.

This is the app's best acquisition surface — it works for people who are not customers.

Story M5.4: Document vault

Given a customer's contracts, receipts, and uploaded documents, **When** they open the vault, **Then** every document is listed, openable, and downloadable through the existing auth-gated URL mutations.

Given a receipt PDF has not been generated yet, **When** the customer requests it, **Then** `requestCustomerReceiptPdf` runs and the screen resolves when it is ready.

Given no network, **When** the vault opens, **Then** previously opened documents are readable from cache.

Story M5.5: Callback request

Given a customer looking at a lot, **When** they request a callback, **Then** a `followUpActions` row is created and appears in the queue staff already work.

Given the request form, **When** it renders, **Then** it asks only what staff need to call back — no lead-capture interrogation.

Story M5.6: Family sharing

Given a customer who is a secondary owner on a family estate, **When** they open My Estate, **Then** they see the estate, its lots, its ceremonies, and its documents — read-only.

Given a secondary owner, **When** they attempt a payment or a reservation on the estate, **Then** it is refused server-side; those remain the primary owner's actions in v1.

Story M5.7: Tributes [Overflow — first to be cut]

Given an occupant record, **When** a contract owner publishes a tribute, **Then** it appears on the occupant's memorial page with their name and dates.

Given a user who is neither a contract owner nor a secondary estate owner, **When** they attempt to publish, **Then** it is refused server-side.

Given a published tribute, **When** staff review it, **Then** they can unpublish it with a logged reason.

Epic M6: Hardening & release [W7–W8]

Story M6.1: Offline reading

Given MNFR3, **When** the app has no connection, **Then** owned lots, contracts, receipts, and the section map render from cache with an explicit "last updated" stamp.

Given a queued write, **When** connectivity returns, **Then** it replays — **except** a reservation, which is never written optimistically offline because it needs a live transaction.

Given ADR-0009 already sets the web cache strategy, **When** the mobile strategy is written, **Then** it follows the same shape rather than inventing a second one.

Story M6.2: Performance budgets in CI

Given MNFR1 and MNFR2, **When** the mobile performance job runs, **Then** cold-start and 3D frame-rate budgets are asserted against the reference device profile and the build fails when they regress.

Given ADR-0016 gates the web app's budgets, **When** the mobile equivalent is added, **Then** it reports in the same place.

Story M6.3: Accessibility pass

Given MNFR7, **When** the audit runs, **Then** every control has a screen-reader label, targets are ≥44×44pt, dynamic type scales to 200% without clipping, and contrast is ≥4.5:1.

Given gold is an accent, **When** any text relies on it to carry meaning, **Then** the audit fails it.

Given a customer using the app in grief, **When** copy is reviewed, **Then** clarity is preferred over cleverness everywhere.

Story M6.4: Security and privacy review

Given MNFR4, **When** every new mobile-facing function is reviewed, **Then** each has `requireRole` as its first awaited line, derives ownership server-side, and emits an audit row.

Given MNFR6, **When** crash reporting is configured, **Then** PII is scrubbed from breadcrumbs and payloads.

Given the existing threat model, **When** the mobile surface is added, **Then** `docs/threat-model.md` covers device token theft, deep-link hijacking, and push payload leakage.

Given RA 10173, **When** onboarding runs, **Then** consent is explicit and data-subject export and deletion route through the existing `dataSubject.ts`.

Story M6.5: Store submission

Given MNFR9, **When** the data-safety and privacy declarations are filed, **Then** they match what the app actually collects — verified against the code, not the intent.

Given spec R7, **When** the build is submitted, **Then** review notes pre-emptively explain that cemetery lots and interment services are real-world goods and services consumed outside the app.

Given a rejection, **When** feedback arrives, **Then** week 8 has room to fix and resubmit.

Story M6.6: Runbook and staff training

Given the runbook covers web operations only, **When** mobile ships, **Then** it gains mobile release, rollback, and push-notification incident response.

Given office staff will work two new queues, **When** training happens, **Then** it is in week 6 alongside the beta — not week 8 alongside launch (spec R8).

Story count and shape

EPIC	STORIES	WEEKS
M0 Release prerequisites	4	W0
M1 Foundation & identity	7	W1–W2
M2 Browse & 3D	6	W2–W3
M3 Reservation	8	W3–W4
M4 Booking	6	W5
M5 Engagement	7 (1 Overflow)	W6
M6 Hardening & release	6	W7–W8
Total	**44**	**8 weeks + W0**

Roughly 15 of the 44 are Convex backend stories, which is the §2.2 finding expressed as a backlog: this is not a project that wraps an existing portal.

De-scope ladder

If week 5 slips, cut in this order:

1. **M5.7** tributes → Phase 2.
2. **M5.6** family sharing → Phase 2.
3. **M2.5** reduces to two 3D views — park overview and lot in situ — dropping the section view.
4. **M4** ships iOS-first; Android follows two weeks later.

Never cut: M3.5 (hold expiry), M3.3 (the hold transaction and its refund path), M6.4 (security and privacy), or the audit emission inside any mutation. These are the ones that cost real families real money.